> **Actualización A01–A18 · 01 nov 2026.** Modelo estrella, soporte, contratos, compuertas, CI y herramientas locales implementados. Los diagnósticos originales se conservan como antecedente; su cierre vigente y evidencias están en Estado de implementación. SQL Server, DAX y Delta cloud siguen pendientes.

PARTE 1 · RESOLUCIÓN TÉCNICA · 50 % / 1.1 · SQL

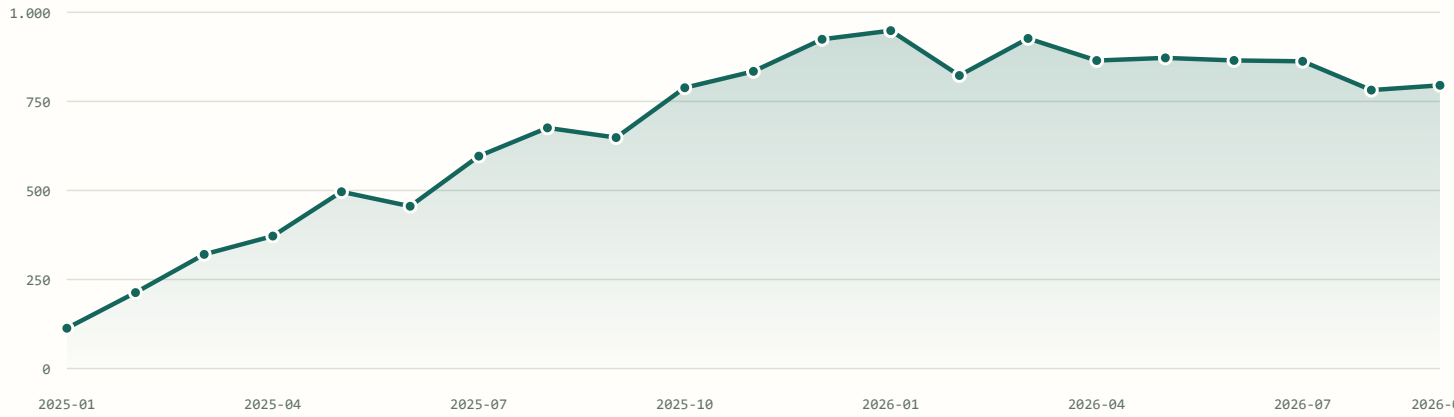
Venta mensual y recompra
con una definición explícita.

QUÉ PIDE LA PRUEBA

Calcular venta neta mensual por canal, variación frente al mes anterior y recompra a 1, 3 y 6 meses por cohorte de registro. Interpretar los resultados en tres líneas.

Total APP WEB TIENDA

Consulta local · 01 nov 2026



TOTAL · 2026-09 · COP 795,1 millones

APP · JUN → SEP 2026

+19,2 %

De $376,5 M a $448,9 M. El crecimiento corresponde a este intervalo concreto.

TIENDA · JUN → SEP 2026

−51,4 %

De $217,6 M a $105,8 M. Una señal prioritaria para investigar.

TOTAL · SEP 2026

$795,1 M

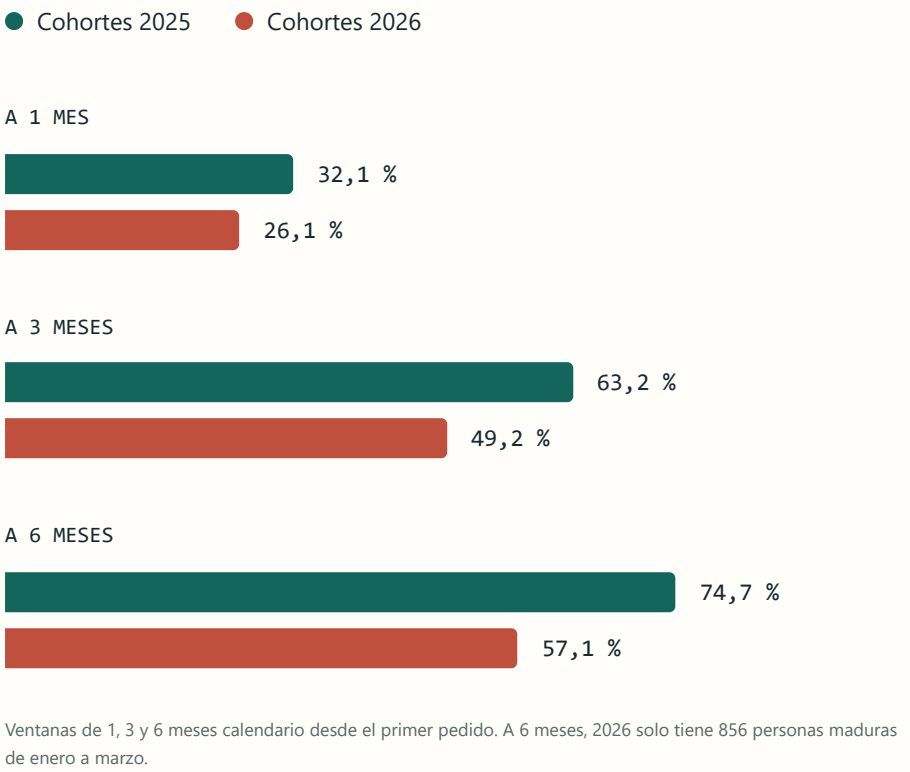
La comparación anual positiva no elimina el estancamiento reciente.

LECTURA CLAVE

Primera decisión: investigar TIENDA por ciudad, categoría y operación. No atribuir la caída a una causa sin contrastarla.

Evidencia · Consulta de solo lectura a data/warehouse.duckdb · 63 filas mensuales · Lectura SQL

Recompra por cohorte de registro



Ventanas de 1, 3 y 6 meses calendario desde el primer pedido. A 6 meses, 2026 solo tiene 856 personas maduras de enero a marzo.

−14,0

puntos porcentuales a 3 meses

La tasa pasa de 63,2 % a 49,2 %. El deterioro persiste usando todos los registrados como base: 60,9 % → 46,4 %.

> **La identidad cambia el análisis.**
> 6.040 registros de clientes representan 6.000 personas al normalizar el documento.

01

Cohorte

Mes de registro de la persona.

02

Primera compra

Primer pedido entregado.

03

Ventana madura

Tiempo suficiente para observar.

04

Recompra

Otro pedido en fecha posterior.

LECTURA CLAVE

TIENDA creció 59,8 % de junio a septiembre de 2025 y cayó 51,4 % en 2026. El patrón no se repite; dos años no bastan para descartar estacionalidad.

Evidencia · Recompra y denominadores · SQL · contraste de TIENDA consultado en Gold

Lectura en tres líneas

1. La venta neta suma COP 14.177.333.600; desde diciembre de 2025 oscila aproximadamente entre $780 y $950 millones mensuales.
2. De junio a septiembre de 2026, TIENDA cae 51,4 % y APP aumenta 19,2 %: la evolución difiere por canal.
3. La recompra a tres meses baja de 63,2 % en cohortes 2025 a 49,2 % en 2026; el dato muestra deterioro, no su causa.

▼ Ver solución T-SQL completa: venta mensual y cohortes

```
-- ============================================================================
-- Parte 1, punto 1 - T-SQL (SQL Server / Fabric Warehouse)
-- Esquema: silver.pedidos, silver.clientes, silver.v_cliente_maestro (ver ddl_tsql.sql)
-- Definiciones:
--   venta neta    = valor_bruto - descuento, solo estado ENTREGADO, por fecha_pedido
--   persona       = id_cliente_maestro (une los clientes duplicados por documento)
--   cohorte       = mes de registro más antiguo de la persona
--   primer pedido = primer pedido ENTREGADO de la persona (por fecha)
--   recompra N    = otro pedido ENTREGADO en fecha posterior y dentro de N meses calendario del primero
```

```sql
--   madurez       = solo cuenta a la persona si primer_pedido + N meses <= última fecha de datos
-- Nota: no se ejecutó contra un SQL Server real; la lógica se validó en DuckDB (ver duckdb/).
-- ========================================================================

-- A) Venta neta mensual por canal con variación frente al mes anterior
;WITH mensual AS (
    SELECT DATEFROMPARTS(YEAR(fecha_pedido_dia), MONTH(fecha_pedido_dia), 1) AS mes,
           canal,
           SUM(valor_neto) AS venta_neta,
           COUNT(*)        AS pedidos
    FROM silver.pedidos
    WHERE estado = 'ENTREGADO'
    GROUP BY DATEFROMPARTS(YEAR(fecha_pedido_dia), MONTH(fecha_pedido_dia), 1), canal
), con_anterior AS (
    SELECT mes, canal, pedidos, venta_neta,
           LAG(venta_neta) OVER (PARTITION BY canal ORDER BY mes) AS venta_mes_anterior
    FROM mensual
)
SELECT mes, canal, pedidos, venta_neta, venta_mes_anterior,
       ROUND(100.0 * (venta_neta - venta_mes_anterior) / NULLIF(venta_mes_anterior, 0), 2) AS variacion_pct
FROM con_anterior
ORDER BY canal, mes;

-- B) Recompra por cohorte de registro a 1, 3 y 6 meses
;WITH personas AS (
    SELECT m.id_cliente_maestro AS persona, MIN(c.cohorte_registro) AS cohorte
    FROM silver.clientes c
    JOIN silver.v_cliente_maestro m ON m.id_cliente = c.id_cliente
    GROUP BY m.id_cliente_maestro
), entregados AS (
    SELECT m.id_cliente_maestro AS persona, p.fecha_pedido_dia AS fecha
    FROM silver.pedidos p
    JOIN silver.v_cliente_maestro m ON m.id_cliente = p.id_cliente
    WHERE p.estado = 'ENTREGADO'
), primer AS (
    SELECT persona, MIN(fecha) AS fecha_primero FROM entregados GROUP BY persona
), ventanas AS (
    SELECT pr.persona, pr.fecha_primero,
           MAX(CASE WHEN e.fecha > pr.fecha_primero AND e.fecha <= DATEADD(MONTH, 1, pr.fecha_primero) THEN 1 ELSE 0 END) AS r1,
           MAX(CASE WHEN e.fecha > pr.fecha_primero AND e.fecha <= DATEADD(MONTH, 3, pr.fecha_primero) THEN 1 ELSE 0 END) AS r3,
           MAX(CASE WHEN e.fecha > pr.fecha_primero AND e.fecha <= DATEADD(MONTH, 6, pr.fecha_primero) THEN 1 ELSE 0 END) AS r6
    FROM primer pr
    JOIN entregados e ON e.persona = pr.persona
    GROUP BY pr.persona, pr.fecha_primero
), limite AS (
    SELECT MAX(fecha_pedido_dia) AS ultima FROM silver.pedidos
)
SELECT p.cohorte,
       COUNT(*)         AS personas_cohorte,
       COUNT(v.persona) AS compradores,
```

```
        SUM(CASE WHEN DATEADD(MONTH, 1, v.fecha_primero) <= l.ultima THEN 1 ELSE 0 END)       AS base_1m,
        SUM(CASE WHEN DATEADD(MONTH, 1, v.fecha_primero) <= l.ultima THEN v.r1  ELSE 0 END)  AS recompra_1m,
        ROUND(100.0 * SUM(CASE WHEN DATEADD(MONTH, 1, v.fecha_primero) <= l.ultima THEN v.r1  ELSE 0 END)
              / NULLIF(SUM(CASE WHEN DATEADD(MONTH, 1, v.fecha_primero) <= l.ultima THEN 1 ELSE 0 END), 0), 2) AS tasa_1m,
        SUM(CASE WHEN DATEADD(MONTH, 3, v.fecha_primero) <= l.ultima THEN 1 ELSE 0 END)       AS base_3m,
        SUM(CASE WHEN DATEADD(MONTH, 3, v.fecha_primero) <= l.ultima THEN v.r3  ELSE 0 END)  AS recompra_3m,
        ROUND(100.0 * SUM(CASE WHEN DATEADD(MONTH, 3, v.fecha_primero) <= l.ultima THEN v.r3  ELSE 0 END)
              / NULLIF(SUM(CASE WHEN DATEADD(MONTH, 3, v.fecha_primero) <= l.ultima THEN 1 ELSE 0 END), 0), 2) AS tasa_3m,
        SUM(CASE WHEN DATEADD(MONTH, 6, v.fecha_primero) <= l.ultima THEN 1 ELSE 0 END)       AS base_6m,
        SUM(CASE WHEN DATEADD(MONTH, 6, v.fecha_primero) <= l.ultima THEN v.r6 ELSE 0 END)  AS recompra_6m,
        ROUND(100.0 * SUM(CASE WHEN DATEADD(MONTH, 6, v.fecha_primero) <= l.ultima THEN v.r6 ELSE 0 END)
              / NULLIF(SUM(CASE WHEN DATEADD(MONTH, 6, v.fecha_primero) <= l.ultima THEN 1 ELSE 0 END), 0), 2) AS tasa_6m
    FROM personas p
    LEFT JOIN ventanas v ON v.persona = p.persona
    CROSS JOIN limite l
    GROUP BY p.cohorte
    ORDER BY p.cohorte;
```

Abrir archivo fuente ↗

Estado actual · A01–A18

SQL local verificado y calendario de meses vacíos implementado. Spark se verifica en CI separado; T-SQL pendiente de ejecución en SQL Server.

PARTE 1 · RESOLUCIÓN TÉCNICA · 50 % / 1.2 · Calidad

Quince problemas.
Un tratamiento para cada uno.

QUÉ PIDE LA PRUEBA

Identificar al menos cinco tipos de problemas, cuantificar los registros afectados y explicar cómo tratarlos.

PEDIDOS

19.734 recibidos



19.270 válidos 464 aislados · 2,35 %

PAGOS

20.883 recibidos



20.124 válidos 759 aislados · 3,63 %

01 · NORMALIZAR

Sin perder el original

810 canales con variantes; 1.200 fechas de pago con dos formatos; documentos que ocultan 40 duplicados.

02 · CONSERVAR Y MARCAR

No borrar una venta real

60 pedidos con cliente inexistente permanecen con una bandera de calidad.

03 · AISLAR

Con motivo y contexto

Duplicados, valores inválidos y fechas incoherentes van a cuarentena con la fila original.

122 pedidos entregados sin pago registrado

345 pagos asociados a pedidos rechazados

LECTURA CLAVE

Estos hallazgos se escalan a negocio. No permiten concluir automáticamente que hay fraude, cartera vencida o doble cobro.

Evidencia · Catálogo de calidad · Reglas · conteos de referencia; una fila puede incumplir varias reglas.

Catálogo completo de la solución

#	Problema	Entidad	Registros	Tratamiento
1	Canal escrito de ocho formas distintas (app, web , PAGINA WEB, TIENDA FISICA...)	pedidos	810	Se normaliza a WEB, APP o TIENDA con un mapa explícito; un valor fuera del mapa va a cuarentena (CANAL_DESCONOCIDO).
2	Fecha de pago en dos formatos (yyyy-mm-dd y dd/mm/yyyy)	pagos	1.200	Se interpreta cada formato de forma explícita; nunca se deja que pandas adivine (03/04/2026 es 3 de abril).
3	Documento del cliente con puntos, espacios o ceros a la izquierda	clientes	40	Se limpia a solo dígitos sin ceros iniciales; se conserva el original en documento_original.
4	Clientes duplicados (mismo documento limpio, distinto id_cliente)	clientes	40	Se conservan ambos ids y la vista v_cliente_maestro los une en una persona para contarla una sola vez.
5	Pedidos con cliente que no existe en clientes	pedidos	60	Se conservan (la venta es real) con flag_cliente_desconocido; el modelo Gold del DDL lo asigna a un cliente DESCONOCIDO.

#	Problema	Entidad	Registros	Tratamiento
6	Pedidos duplicados con otro id_pedido (mismo cliente, fecha-hora, valor y categoría)	pedidos	295	Se conserva uno (el que ya estaba en Silver, o el menor id) y el resto va a cuarentena.
7	Descuento negativo o mayor que el valor bruto	pedidos	74	Cuarentena (DESCUENTO_INVALIDO).
8	Valor bruto cero o negativo	pedidos	35	Cuarentena (VALOR_BRUTO_NO_POSITIVO).
9	Fecha de pedido fuera del rango del caso (2027)	pedidos	20	Cuarentena (FECHA_FUERA_DE_RANGO).
10	Entrega anterior a la fecha del pedido	pedidos	87	Cuarentena (ENTREGA_ANTERIOR_A_PEDIDO).
11	Pagos duplicados (mismo pedido, valor y medio; incluso con la fecha en otro formato)	pagos	344	Se conserva uno y el resto va a cuarentena. El conteo es sobre los pagos que pasaron las demás reglas; sobre el archivo completo, sin filtrar, son más (350 con los datos del caso).
12	Pagos de un pedido que no existe	pagos	70	Cuarentena (PAGO_HUERFANO).
13	Pagos de un pedido que fue rechazado	pagos	345	Cuarentena (PEDIDO_EN_CUARENTENA); si el pedido se corrige o llega después, el pago se relee y se carga en la siguiente corrida.
14	Ticket con fecha de cierre anterior a la de creación	tickets	99	Tratamiento propuesto (el ETL actual no carga tickets; el DDL lo prevé): se conserva el ticket y horas_resolucion queda nula para no promediar tiempos negativos.
15	Ticket cerrado sin fecha de cierre	tickets	59	Tratamiento propuesto, igual que el anterior; tampoco está implementado en el ETL.

Estado actual · A01–A18

Contratos CSV, cuarentena e incidencias abiertas/resueltas implementados localmente. Umbrales y responsables de negocio pendientes de acordar.

PARTE 1 · RESOLUCIÓN TÉCNICA · 50 % / 1.3 · Python / ETL

Un ETL que se puede *volver a ejecutar.*

QUÉ PIDE LA PRUEBA

Cargar pedidos y pagos de forma incremental e idempotente, aplicar reglas, aislar inválidos y generar venta neta diaria por canal y categoría. Incluir al menos tres pruebas.

01

CSV

Clientes · pedidos · pagos

02

Bronze / Parquet

Snapshot crudo · SHA-256 · run_id

03

Silver / DuckDB

Tipos · deduplicación · row_hash

04

Gold / DuckDB

Día × canal × categoría

↳ **Cuarentena**

Registro + regla + hash + JSON original

↳ **Control**

Watermark + historial de ejecuciones

↳ **Consumo**

SQL analítico · reporte · modelo propuesto

TRANSACCIÓN DUCKDB · Silver + cuarentena + watermark + Gold se confirman juntos

TRAZABILIDAD

Un archivo, una huella

El manifiesto evita aterrizar dos veces el mismo contenido; Silver lee la copia Bronze.

GRANO ANALÍTICO

7.925 filas Gold

16.855 pedidos entregados agregados por fecha, canal y categoría.

PORTABILIDAD

Reglas en PySpark

Existe implementación paralela y notebook Delta. Ejecución en nube todavía pendiente.

LECTURA CLAVE

El agregado diario local sí existe. El modelo estrella y el modelo semántico de la plataforma necesitan carga y validación propias.

Evidencia · pipeline.py · bronze.py · store.py

Incremental, idempotencia y recuperación

COMPORTAMIENTO DEL DISEÑO

Sin cambios, sin duplicados

`Mismo SHA-256 → mismo Bronze → mismo row_hash`

El manifiesto reutiliza el snapshot. Silver no reescribe filas idénticas; la llave de cuarentena evita repetir incidencias.

Resultado esperado: 0 insertados y 0 actualizados.



LECTURA CLAVE

Pedidos: lectura completa y escritura por hash. Pagos: filtro por fecha_carga más tres días, después de leer el snapshot. Es incremental en procesamiento/escritura, no una extracción CDC de origen.

Evidencia · Pruebas de idempotencia, cambios y rollback · Límites del incremental

Pruebas que respaldan el comportamiento

Caso	Qué verifica
Dos ejecuciones iguales	Mismos datos en Silver, cuarentena y Gold; sin duplicación.
Incremento de pedidos y pagos	Solo se escribe lo nuevo o modificado.
Cambio de estado o fecha	Se corrigen los días afectados en Gold.
Fallo entre Silver y Gold	Rollback de la transacción local.
Pago antes del pedido	Revalidación cuando llega un pedido válido.

Ejecución desde cero

```
pip install -r requirements.txt
python scripts/cargar_datos.py --desde-cero
python scripts/cargar_datos.py
python -m pytest -q
```

La primera orden de carga reconstruye la base y Bronze: usarla en el entorno local del caso con los CSV en data/original. La segunda permite comprobar idempotencia.

▼ Ver orquestador Python completo

```
"""Orquestación: clientes -> pedidos -> pagos -> venta neta diaria.

Orden importa: los pagos se validan contra los pedidos ya cargados, y los pedidos
contra los clientes. Toda la corrida es una sola transacción: si el proceso se cae a la
mitad no queda nada a medias (Silver sin Gold) y basta con volver a ejecutarlo. Lo único
que sobrevive a una caída es el Parquet de Bronze, que se vuelve a escribir igual.
"""
from __future__ import annotations

import uuid
from datetime import datetime, timedelta
from pathlib import Path

import pandas as pd

from . import bronze, rules, store, tickets
from .operations import bloqueo, actualizar_incidencias, validar_publicacion
from .star import refrescar_estrella
from .config import ESTADO_VENTA, Config


def ejecutar(raw_dir, db_path, cfg=None, bronze_dir=None, recalcular_gold=False,
             politica=None, reconciliar=False):
    """Escritor exclusivo local. Reconciliar relee pagos, nunca infiere borrados."""
    with bloqueo(db_path):
        return _ejecutar(raw_dir, db_path, cfg, bronze_dir, recalcular_gold, politica, reconciliar)
```

```
def _ejecutar(raw_dir: str | Path, db_path: str | Path, cfg: Config | None = None,
              bronze_dir: str | Path | None = None, recalcular_gold: bool = False, politica=None, reconciliar: bool = False) -> dict:
    """recalcular_gold: recalcula todos los días de Gold, no solo los tocados. Hace falta
    cuando cambia la definición de venta (por ejemplo, atribuir a fecha_entrega) sin que
    cambien los datos."""
    cfg = cfg or Config()
    raw_dir = Path(raw_dir)
    bronze_dir = Path(bronze_dir) if bronze_dir else Path(db_path).parent / "bronze"
    run_id = str(uuid.uuid4())
    inicio = datetime.now()
    con = store.conectar(str(db_path))
    resumen: dict = {"run_id": run_id, "bronze": {}}

    def fuente(tabla: str) -> pd.DataFrame:
        # CSV -> Bronze (Parquet crudo, inmutable) -> Silver. Silver nunca lee el CSV directo.
        df, info = bronze.aterrizar(con, raw_dir, bronze_dir, tabla, run_id)
        resumen["bronze"][tabla] = info
        return df

    con.begin()
    try:
        # ---- clientes (dimensión pequeña: se lee completa; el hash evita reescrituras)
        cli_raw = fuente("clientes")
        cli_ok, cli_q = rules.validar_clientes(cli_raw, cfg)
        resumen["clientes"], _ = store.upsert(con, "silver_clientes", cli_ok)
        resumen["clientes"]["leidos"] = len(cli_raw)
        resumen["clientes"]["cuarentena_nueva"] = store.registrar_cuarentena(con, cli_q, run_id)
        store.retirar(con, "silver_clientes", _rechazados(cli_q, cli_ok, "id_cliente"))
        actualizar_incidencias(con, 'clientes', cli_raw, cli_q, run_id)
        resumen['clientes']['rechazados_lote'] = len(cli_raw) - len(cli_ok)
        clientes_ids = set(con.execute("SELECT id_cliente FROM silver_clientes").df()["id_cliente"])

        # ---- pedidos: la fuente no trae fecha de carga, así que el incremental se hace
        # por contenido: solo se escribe lo nuevo o lo que cambió de hash.
        ped_raw = fuente("pedidos")
        ped_ok, ped_q = rules.validar_pedidos(
            ped_raw, cfg, clientes_ids, store.negocio_existente(con, "silver_pedidos"))
        dias_antes = con.execute("SELECT id_pedido, fecha_pedido_dia FROM silver_pedidos").df()
        resumen["pedidos"], ped_escritos = store.upsert(con, "silver_pedidos", ped_ok)
        resumen["pedidos"]["leidos"] = len(ped_raw)
        resumen["pedidos"]["cuarentena_nueva"] = store.registrar_cuarentena(con, ped_q, run_id)
        # Un pedido que ya estaba en Silver y hoy es inválido (dato corregido en la fuente,
        # regla nueva, rango más estrecho) sale de Silver y deja de sumar venta.
        ped_retirados = store.retirar(con, "silver_pedidos", _rechazados(ped_q, ped_ok, "id_pedido"))
        resumen["pedidos"]["retirados"] = len(ped_retirados)

        actualizar_incidencias(con, 'pedidos', ped_raw, ped_q, run_id)
        resumen['pedidos']['rechazados_lote'] = len(ped_raw) - len(ped_ok)
```

```
            con.execute("UPDATE silver_pedidos SET flag_cliente_desconocido = NOT EXISTS "
                        "(SELECT 1 FROM silver_clientes c WHERE c.id_cliente=silver_pedidos.id_cliente)")

            # Días de venta a recalcular: el día nuevo de cada fila escrita y, si la fila ya
            # existía, también su día anterior (por si el pedido cambió de fecha), más los días
            # de los pedidos retirados.
            fechas_afectadas = _fechas_afectadas(ped_escritos, dias_antes, ped_retirados)

            # ---- pagos: incremental por watermark de fecha_carga, con ventana de reproceso
            pag_raw = fuente("pagos")
            wm = store.leer_watermark(con, "pagos")
            if wm is not None and not reconciliar:
                desde = pd.Timestamp(wm) - timedelta(days=cfg.lookback_dias)
                carga = rules._a_fecha(pag_raw["fecha_carga"])
                # Los pagos que esperan a su pedido se releen siempre: si el pedido llega después
                # de que el pago salió de la ventana, de otro modo quedarían huérfanos para siempre.
                pendiente = pag_raw["id_pago"].astype(str).str.strip().isin(store.pagos_pendientes(con))
                pag_raw = pag_raw[(carga >= desde) | carga.isna() | pendiente].reset_index(drop=True)
            pedidos_dia = con.execute(
                "SELECT id_pedido, fecha_pedido_dia FROM silver_pedidos").df().set_index("id_pedido")["fecha_pedido_dia"]
            pedidos_dia = pd.to_datetime(pedidos_dia)
            pag_ok, pag_q = rules.validar_pagos(
                pag_raw, cfg, pedidos_dia, store.ids_en_cuarentena(con, "pedidos"),
                store.negocio_existente(con, "silver_pagos"))
            resumen["pagos"], _ = store.upsert(con, "silver_pagos", pag_ok)
            resumen["pagos"]["leidos"] = len(pag_raw)
            resumen["pagos"]["cuarentena_nueva"] = store.registrar_cuarentena(con, pag_q, run_id)
            resumen["pagos"]["retirados"] = len(store.retirar(con, "silver_pagos", _rechazados(pag_q, pag_ok, "id_pago")))
            actualizar_incidencias(con, 'pagos', pag_raw, pag_q, run_id)
            resumen['pagos']['rechazados_lote'] = len(pag_raw) - len(pag_ok)
            if len(pag_raw):
                maximo = rules._a_fecha(pag_raw["fecha_carga"]).max()
                if pd.notna(maximo):
                    previo = pd.Timestamp(wm) if wm is not None else maximo
                    store.escribir_watermark(con, "pagos", max(maximo, previo).date(), run_id)

            # Soporte es opcional y no altera el cálculo de venta.
            con.execute(tickets.ESQUEMA)
            if (raw_dir / 'tickets_soporte.csv').exists():
                ticket_raw = fuente('tickets_soporte')
                ticket_ok, ticket_q = tickets.validar_tickets(ticket_raw)
                resumen['tickets_soporte'], _ = store.upsert(con, 'silver_tickets', ticket_ok)
                store.retirar(con, 'silver_tickets', _rechazados(ticket_q, ticket_ok, 'id_ticket'))
                resumen['tickets_soporte'].update(leidos=len(ticket_raw),
                    rechazados_lote=len(ticket_raw)-len(ticket_ok),
                    cuarentena_nueva=store.registrar_cuarentena(con, ticket_q, run_id))
                actualizar_incidencias(con, 'tickets_soporte', ticket_raw, ticket_q, run_id)

            # ---- gold: venta neta diaria solo para los días tocados (o todos, si se pide)
            if recalcular_gold:
```

```
            fechas_afectadas = store.dias_con_venta(con)
        resumen["dias_recalculados"] = store.refrescar_venta_neta(con, fechas_afectadas, ESTADO_VENTA)

        resumen['publicacion'] = validar_publicacion(con, resumen, politica)
        resumen['estrella'] = refrescar_estrella(con, cfg)
        con.execute("CREATE TABLE IF NOT EXISTS publicacion_actual "
                    "(id INTEGER PRIMARY KEY, run_id VARCHAR, validado_en TIMESTAMP, venta_neta BIGINT)")
        con.execute("INSERT OR REPLACE INTO publicacion_actual VALUES (1,?,current_timestamp,?)",
                    [run_id, resumen['publicacion']['venta_neta']])
        _log(con, run_id, inicio, "OK", resumen)
        con.commit()
    except Exception:
        con.rollback()
        _log(con, run_id, inicio, "FALLIDO", resumen)
        raise
    finally:
        con.close()
    return resumen


def _rechazados(cuarentena: pd.DataFrame, validos: pd.DataFrame, clave: str) -> set[str]:
    """Ids que en esta corrida fueron a cuarentena y no quedaron entre los válidos."""
    ids = {i.strip() for i in cuarentena["id_registro"] if i.strip()}
    return ids - set(validos[clave])


def _fechas_afectadas(escritos: pd.DataFrame, dias_antes: pd.DataFrame,
                      retirados: pd.DataFrame | None = None) -> list:
    dias = set()
    if not escritos.empty:
        dias |= set(pd.to_datetime(escritos["fecha_pedido_dia"]).dt.date)
        previos = dias_antes[dias_antes["id_pedido"].isin(escritos["id_pedido"])]
        dias |= set(pd.to_datetime(previos["fecha_pedido_dia"]).dt.date)
    if retirados is not None and not retirados.empty:
        dias |= set(pd.to_datetime(retirados["fecha_pedido_dia"]).dt.date)
    return sorted(dias)


def _log(con, run_id, inicio, estado, resumen):
    def suma(clave):
        return sum(resumen.get(t, {}).get(clave, 0) for t in ("clientes", "pedidos", "pagos", "tickets_soporte"))

    con.execute(
        "INSERT INTO etl_run_log VALUES (?,?,?,?,?,?,?,?,?,?)",
        [run_id, inicio, datetime.now(), estado, suma("leidos"),
         suma("insertados") + suma("actualizados") + suma("sin_cambios"),
         suma("cuarentena_nueva"), suma("insertados"), suma("actualizados"), suma("sin_cambios")],
    )
```

Abrir archivo fuente ↗

Estado actual · A01–A18

ETL local ampliado con soporte, modelo estrella, conciliación antes de publicar, bloqueo y recuperación. 116 pruebas locales pasan; Spark se verifica por separado. Orquestación Delta pendiente de validación.

PARTE 1 · RESOLUCIÓN TÉCNICA · 50 % / 1.4 · DAX

Venta, ticket y variación.
El contexto importa.

QUÉ PIDE LA PRUEBA

Escribir y explicar venta neta, ticket promedio y variación frente al mismo mes del año anterior.

CONTRATO DEL CASO

Σ (bruto − descuento)

`estado = ENTREGADO`

Ticket promedio = venta neta / pedidos entregados

Fecha de atribución: fecha_pedido.
2025-01-01 → 2026-09-30.

Fecha | Canal

Categoría | Cliente

↓ relaciones por validar ↓

fact_pedido

un pedido por fila · DDL propuesto

↓

Medidas DAX → modelo certificado

COMPARACIÓN CORRECTA

Ene–sep vs ene–sep

Para 2026: $7.739,1 M frente a $3.891,3 M en 2025 → +98,88 %.

COMPARACIÓN ENGAÑOSA

9 meses vs 12 meses

Usar todo 2025 como base daría +20,2 %. El período comparable importa.

SIN BASE ANTERIOR

Vacío, no cero

En 2025 no hay año previo disponible. DAX debe devolver BLANK.

LECTURA CLAVE

Brecha crítica: el ETL local genera el agregado diario, pero no carga el esquema estrella del DDL. Falta construir ese puente antes de validar DAX y el reporte ODBC.

Evidencia · medidas.dax · Valores esperados · DDL

▼ Ver todas las medidas DAX y la columna tiene_datos

```
-- ========================================================================
-- Parte 1, punto 4 - Medidas DAX (modelo semántico sobre la capa Gold)
-- Modelo: gold.fact_pedido (un pedido por fila) -> gold.dim_fecha por fecha_key (muchos a uno).
-- dim_fecha[fecha] marcada como tabla de fechas. La venta se atribuye a fecha_pedido.
-- Estas medidas no se ejecutaron en Power BI: sus valores esperados salen del cálculo
-- equivalente en SQL (ver docs/03_dax.md) y sirven para comprobarlas.
-- ========================================================================

Venta Neta =
CALCULATE (
    SUMX ( fact_pedido, fact_pedido[valor_bruto] - fact_pedido[descuento] ),
    fact_pedido[estado] = "ENTREGADO"
)

Pedidos Entregados =
CALCULATE ( COUNTROWS ( fact_pedido ), fact_pedido[estado] = "ENTREGADO" )

Ticket Promedio =
DIVIDE ( [Venta Neta], [Pedidos Entregados] )

-- Columna calculada en dim_fecha: TRUE hasta el último día con pedidos (2026-09-30).
tiene_datos =
```

```
    dim_fecha[fecha_key] <= MAX ( fact_pedido[fecha_key] )

    -- Solo se desplazan las fechas que tienen datos: con el año 2026 seleccionado compara
    -- enero-septiembre de 2026 contra enero-septiembre de 2025, no contra 2025 completo.
    Venta Neta Mismo Mes Año Anterior =
    CALCULATE (
        [Venta Neta],
        CALCULATETABLE ( DATEADD ( dim_fecha[fecha], -1, YEAR ), dim_fecha[tiene_datos] = TRUE () )
    )

    Variación vs Mismo Mes Año Anterior =
    VAR actual = [Venta Neta]
    VAR previo = [Venta Neta Mismo Mes Año Anterior]
    RETURN
        IF ( NOT ISBLANK ( previo ), DIVIDE ( actual - previo, previo ) )
    -- Formato de la medida: porcentaje con un decimal (0.0%).
```

Abrir archivo fuente ↗

Medida	Cómo funciona
Venta Neta	CALCULATE filtra ENTREGADO; SUMX agrega bruto menos descuento.
Ticket Promedio	DIVIDE usa venta neta y cantidad de pedidos entregados; sin pedidos devuelve vacío.
Venta Neta Mismo Mes Año Anterior	DATEADD desplaza un año el contexto; tiene_datos evita comparar nueve meses contra doce.
Variación anual	(actual − previo) / previo. Sin período anterior, devuelve BLANK.

Estado actual · A01–A18

Modelo estrella gold.* construido y probado localmente. Medidas DAX preparadas; relaciones y resultados pendientes de verificar en Power BI.

Menos trabajo repetido.
Mejores rutas de acceso.

QUÉ PIDE LA PRUEBA

Identificar problemas, reescribir la consulta lenta y proponer índices o particiones para 50 millones de pedidos.

ABLACIÓN · 100.000 PEDIDOS · SQLITE



Misma salida en las variantes del experimento documentado. Resultados dependientes del equipo y del conjunto sintético.

DEL FILTRO A LA BÚSQUEDA

```
YEAR(fecha_pedido) = ...
CAST(t.id_cliente AS VARCHAR) = ...

↓

fecha_pedido >= @inicio
AND fecha_pedido < @fin
t.id_cliente = c.id_cliente
```

01

Agregar una vez

Evitar trabajo correlacionado repetido.

02

Filtrar por rango

Conservar la columna de fecha sin funciones.

03

Cruzar tipos iguales

Eliminar conversiones innecesarias.

04

Validar en destino

Plan real · IO · CPU · equivalencia.

LECTURA CLAVE

Pendiente: benchmark representativo en SQL Server, incluidos costo de escritura de índices, sesgo de datos y parámetros. No hay prueba de rendimiento sobre 50 millones en ese motor.

Equivalencia antes que cambio de negocio

La reescritura conserva todos los estados, como la consulta original. Filtrar ENTREGADO cambia la definición y debe hacerse por decisión explícita. EXISTS expresa la pertenencia; sustituir IN/DISTINCT no garantiza por sí solo una mejora. Los empates de fecha en el último ticket requieren acordar un criterio determinista.

Índice / diseño	Objetivo
pedidos(fecha_pedido) INCLUDE (id_cliente, valor_bruto, descuento)	Leer el mes por rango con columnas cubiertas.
pedidos(id_cliente) WHERE estado = 'DEVUELTO'	Resolver pertenencia con un índice filtrado.
tickets_soporte(id_cliente, fecha_creacion DESC) INCLUDE (canal)	Buscar el último ticket por cliente.
Partición mensual por fecha_pedido	Facilitar mantenimiento y poda cuando el plan lo permita; medir costo de escritura y alineación.

▼ Ver reescritura T-SQL, índices y esquema de particiones

```
-- ============================================================================
-- Parte 1, punto 5 - Optimización de la consulta de 50 millones de pedidos (T-SQL)
-- Nota: no se ejecutó en SQL Server. La forma del problema se midió con
-- experimento_optimizacion.py (SQLite, datos sintéticos); ver docs/05_optimizacion.md.
-- ============================================================================

/* ----------------------------------------------------------------------------
   PROBLEMAS DE LA CONSULTA ORIGINAL
   1. Subconsulta correlacionada para venta_mes: se ejecuta una vez por cliente.
   2. YEAR(p.fecha_pedido) / MONTH(p.fecha_pedido): función sobre la columna; el índice por
      fecha no se puede usar (predicado no SARGable) y se lee la tabla completa.
   3. CAST(... AS VARCHAR(20)) en ambos lados del cruce con tickets_soporte: conversión
      sobre las dos columnas, ninguna búsqueda por índice. Ambos id_cliente ya son VARCHAR(9)
      (ver DDL): el CAST no aporta nada y es lo que más pesa.
   4. IN (SELECT DISTINCT ...): buena práctica, no causa de lentitud. El optimizador suele
      convertirlo en semiunión y descartar el DISTINCT; EXISTS expresa mejor la intención.
   5. TOP 1 ... ORDER BY fecha_creacion DESC sin índice (id_cliente, fecha_creacion DESC):
      ordena los tickets del cliente en cada ejecución.
   6. GETDATE(): SQL Server lo evalúa una vez por consulta, no por fila; se pasa a una
      variable para que el rango del mes sea explícito y se pueda probar con otra fecha.
   7. Observación de negocio: el original suma TODOS los estados del mes (también devueltos y
      cancelados). La reescritura conserva ese resultado para ser equivalente; si la intención
      es venta neta, añade AND p.estado = 'ENTREGADO' (se señala abajo).
---------------------------------------------------------------------------- */

-- CONSULTA REESCRITA (mismo resultado que la original)
DECLARE @ini DATE = DATEFROMPARTS(YEAR(GETDATE()), MONTH(GETDATE()), 1);
DECLARE @fin DATE = DATEADD(MONTH, 1, @ini);

WITH venta AS (
    SELECT p.id_cliente,
           SUM(p.valor_bruto - p.descuento) AS venta_mes
    FROM dbo.pedidos AS p
    WHERE p.fecha_pedido >= @ini
      AND p.fecha_pedido <  @fin          -- rango SARGable: usa el índice por fecha
      -- AND p.estado = 'ENTREGADO'       -- descomentar si se quiere venta neta (ver punto 7)
    GROUP BY p.id_cliente
)
SELECT c.id_cliente,
       v.venta_mes,
       u.canal AS ultimo_canal_soporte
FROM dbo.clientes AS c
LEFT JOIN venta AS v ON v.id_cliente = c.id_cliente
OUTER APPLY (
    SELECT TOP (1) t.canal
    FROM dbo.tickets_soporte AS t
    WHERE t.id_cliente = c.id_cliente     -- mismo tipo a ambos lados: búsqueda por índice
    ORDER BY t.fecha_creacion DESC
```

```
) AS u
WHERE EXISTS (
    SELECT 1 FROM dbo.pedidos AS d
    WHERE d.id_cliente = c.id_cliente AND d.estado = 'DEVUELTO'
)
ORDER BY v.venta_mes DESC;
GO

-- ÍNDICES
-- Ventas del mes: lectura por rango de fechas, sin ir a la tabla (cubre todas las columnas).
CREATE INDEX ix_pedidos_fecha
    ON dbo.pedidos (fecha_pedido)
    INCLUDE (id_cliente, valor_bruto, descuento);

-- Pedidos devueltos: índice filtrado, pequeño, solo con las filas que interesan al EXISTS.
CREATE INDEX ix_pedidos_devueltos
    ON dbo.pedidos (id_cliente)
    WHERE estado = 'DEVUELTO';

-- Último canal de soporte: la búsqueda del TOP 1 entra directo a la fila más reciente.
CREATE INDEX ix_tickets_cliente_fecha
    ON dbo.tickets_soporte (id_cliente, fecha_creacion DESC)
    INCLUDE (canal);
GO

-- PARTICIONES (50M de filas)
-- Partición mensual por fecha_pedido: la consulta del mes en curso toca una sola partición
-- (eliminación de particiones) y el archivado de meses antiguos es un SWITCH, no un DELETE.
-- Es un beneficio sobre todo de mantenimiento; la velocidad de esta consulta la dan los índices.
CREATE PARTITION FUNCTION pf_pedidos_mes (DATETIME2)
AS RANGE RIGHT FOR VALUES ('2025-01-01', '2025-02-01', '2025-03-01' /* ... un valor por mes ... */);
CREATE PARTITION SCHEME ps_pedidos_mes
AS PARTITION pf_pedidos_mes ALL TO ([PRIMARY]);
-- La tabla pasa a usar: CREATE CLUSTERED INDEX cx_pedidos ON dbo.pedidos (fecha_pedido, id_pedido)
--                       ON ps_pedidos_mes (fecha_pedido);
-- Los índices anteriores se crean alineados (ON ps_pedidos_mes (fecha_pedido)).
-- Alternativa para analítica: índice columnstore no agrupado sobre pedidos.
GO
```

Abrir archivo fuente ↗

Estado en la entrega inicial · avances A01–A18 en el enlace superior

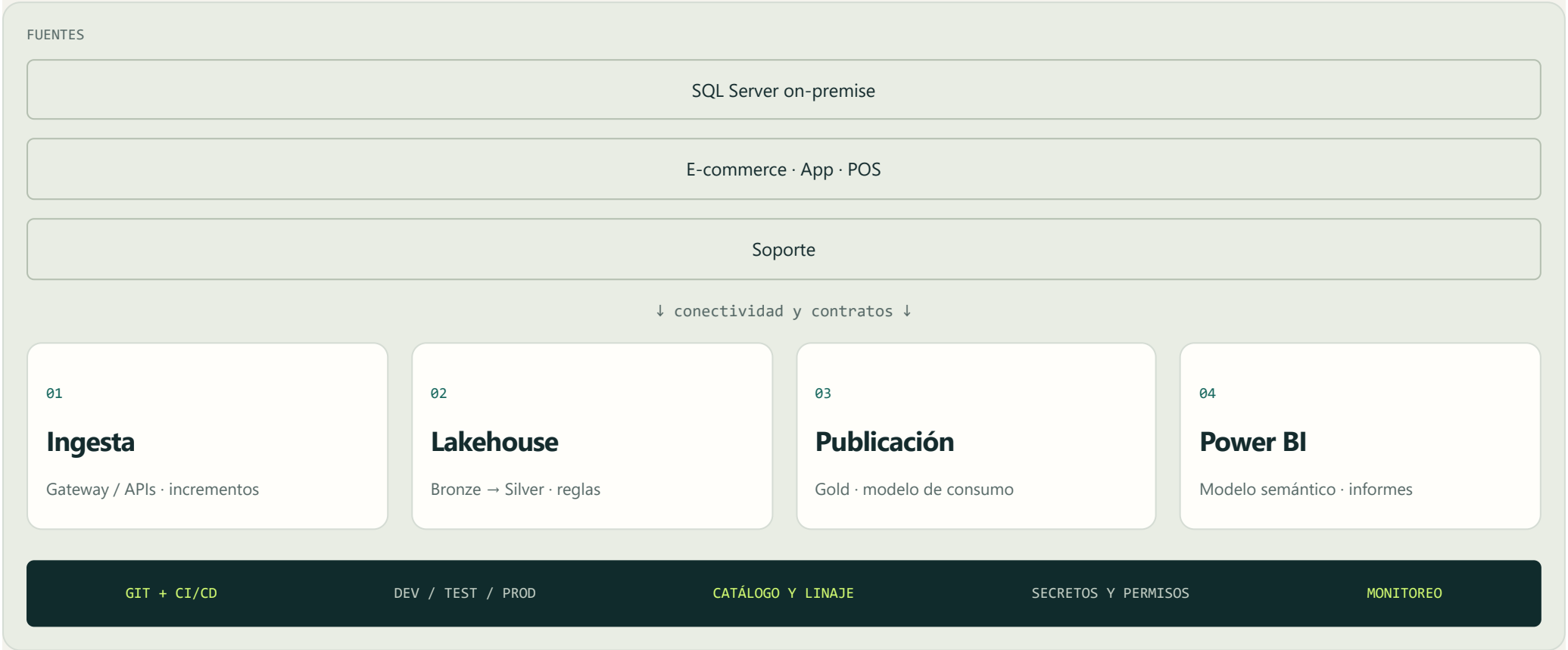
Experimento documentado en SQLite con datos sintéticos. No se midió SQL Server con 50 millones de pedidos. DDL de particiones ilustrativo, pendiente completar límites y validar restricciones del destino.

Manuel Alberto Coy Benavides · Solución del proyecto Andina Market · Presentación asistida por Codex · Revisión 01 nov 2026

Una arquitectura *para los próximos 12 meses.*

QUÉ PIDE LA PRUEBA

Proponer un diagrama de arquitectura de datos a 12 meses.



HIPÓTESIS PREFERIDA

Microsoft Fabric

Probar el recorrido completo con los informes prioritarios y medir operación, conciliación y costo. La experiencia del equipo en SQL/BI es un supuesto por validar.

ALTERNATIVA

Databricks

Comparar con la misma carga si ingeniería, ML, streaming o requisitos multicloud justifican otra plataforma. No decidir solo por volumen.

LECTURA CLAVE

La prueba local reduce incertidumbre sobre reglas. No prueba conectividad, permisos, MERGE Delta, capacidad, costos ni operación de ninguna nube.

Evidencia · Arquitectura original de dos páginas · Notebook generado · sin afirmaciones de precios o licencias vigentes.

Meses 1–3

Ventas conciliadas de Bronze a Gold.

Meses 4–6

Ampliar fuentes y dominios.

Meses 7–9

Migrar informes.

Meses 10–12

Gobierno, ML e IA.

La arquitectura resumida de dos páginas sigue disponible en 04_arquitectura.pdf. Esta pestaña es el apoyo para la sustentación.

Estado en la entrega inicial · avances A01–A18 en el enlace superior

Arquitectura objetivo propuesta; no desplegada. El prototipo local y las reglas PySpark aportan una base, no validan servicios, permisos ni integración de plataforma.

Manuel Alberto Coy Benavides · Solución del proyecto Andina Market · Presentación asistida por Codex · Revisión 01 nov 2026

Fabric como punto de partida.
La decisión se valida.

QUÉ PIDE LA PRUEBA

Recomendar entre dos alternativas con criterios de costo, riesgo y capacidad del equipo.

RECOMENDACIÓN DEL PROYECTO

Microsoft Fabric

El caso ya tiene SQL Server y más de 30 informes de Power BI. Propongo comprobar un recorrido integrado de ingesta a modelo semántico con un piloto de ventas.

Criterio	Fabric: hipótesis inicial	Databricks: alternativa	Qué validar
Costo	Evaluar capacidad compartida y licencias de consumo de BI.	Evaluar cómputo, almacenamiento, integración y BI.	Cotizar la misma carga, usuarios y horarios; no hay presupuesto medido.
Riesgo	Concentración en ecosistema Microsoft y dependencia de sus servicios.	Más componentes de integración según el diseño elegido.	Piloto con conectividad, seguridad, fallos y recuperación.
Equipo	Puede encajar con la base de SQL Server y Power BI.	Evaluar experiencia en Spark, Python y operación de plataforma.	Inventario real de capacidades; el perfil del equipo es un supuesto.

Criterio	Fabric: hipótesis inicial	Databricks: alternativa	Qué validar
Estrategia	Priorizar el modelo certificado y los informes existentes.	Reevaluar si ML, streaming o requisitos multicloud son determinantes.	Usar criterios de éxito acordados, no solo volumen de datos.

01

Misma carga

Comparar datos, concurrencia y uso.

02

Mismo resultado

Conciliación y frescura exigidas.

03

Costo completo

Licencias, operación y formación.

04

Decisión

Elegir con evidencia del piloto.

Cuándo revisaría la elección

Si las necesidades de ingeniería, ML, streaming o portabilidad pesan más que la integración de BI, contrastaría Databricks con los mismos criterios. Parquet, Delta y reglas Spark reducen parte del costo de migración, pero no eliminan el trabajo de integración.

Estado en la entrega inicial · avances A01–A18 en el enlace superior

Recomendación arquitectónica del proyecto, no cotización ni comparación comercial actualizada. Costos, condiciones de licencia y capacidad del equipo pendientes de confirmar.

Versionar, desplegar
y saber cuándo detenerse.

QUÉ PIDE LA PRUEBA

Explicar Git, ambientes, CI/CD, monitoreo, alertas y calidad de datos.

Carga terminada

Control técnico

↓

¿Concilia?

Volumen · calidad · frescura

↓

SÍ → Publicar versión validada

NO → Retener + alertar + investigar

ANTES DEL CAMBIO

PR → pruebas → Test

Versionar DDL, notebooks, reglas y modelo. Proteger la promoción a producción.

DURANTE LA CARGA

Medir y reconciliar

Definir frescura, duración, filas, rechazos, totales y umbrales con historial real.

DESPUÉS DEL FALLO

Recuperar y comunicar

Runbook, responsable, última versión válida y ensayo de restauración.

LECTURA CLAVE

Falta implementar CI/CD, programación, alertas, permisos, retención, respaldos y recuperación. Un git revert revierte código; no revierte automáticamente datos, esquemas ni modelos publicados.

Evidencia · Diseño de operación · Registro local de corridas · diagnóstico de archivos disponibles.

Práctica	Resolución propuesta	Evidencia de aceptación
Git	Versionar SQL, DDL, notebook generador, reglas, modelos y procedimientos; PR revisado.	Historial y políticas aplicadas.
Ambientes	Dev/Test/Prod separados, secretos externos y datos de prueba enmascarados.	Permisos y aislamiento comprobados.
CI/CD	Pruebas de reglas y contratos; despliegue a Test; aprobación para Prod.	Cambio correcto promovido y cambio defectuoso bloqueado.
Calidad y alertas	Frescura, volumen, cuarentena, conciliación y duración; aviso con responsable.	Fallo simulado evita publicar y genera alerta.
Recuperación	Versión válida anterior, reejecución controlada y restauración.	Ensayo con RPO/RTO acordados y medidos.

Estado en la entrega inicial · avances A01–A18 en el enlace superior

Existen pruebas y log de corridas local. Git, CI/CD, compuertas, alertas y recuperación operativa se proponen; no se declaran implementados.

Manuel Alberto Coy Benavides · Solución del proyecto Andina Market · Presentación asistida por Codex · Revisión 01 nov 2026

Entregables por fase.
Una cifra para todas las áreas.

QUÉ PIDE LA PRUEBA

Definir los primeros 90 días y cómo garantizar una única cifra de ventas.

01–30

Acordar y conectar

Inventario de informes, contrato firmado, dueños por dominio, repositorio y ambientes.

Salida: definición aprobada + ingesta trazable.

31–60

Limpiar y conciliar

Silver, cuarentena, carga del modelo destino y Gold contra el transaccional.

Salida: diferencia cero o explicada y aceptada.

61–90

Certificar y adoptar

Validar DAX, migrar informes prioritarios, activar alertas y practicar recuperación.

Meta propuesta: 8 informes en el modelo oficial.

Meses 4—6

Más fuentes y dominios

Meses 7—9

Migración del resto de informes

Meses 10—12

Gobierno, ML e IA con evidencia

LECTURA CLAVE

El primer hito no es “elegir una nube”. Es lograr que Finanzas y Comercial acepten la misma cifra y puedan rastrear cualquier diferencia.

Evidencia · Plan original · metas y responsables propuestos, sujetos a capacidad y presupuesto.

Mecanismo de cifra única

01

Acordar

Finanzas + Comercial firman definición y atribución.

02

Versionar

Un dueño aprueba cambios de contrato.

03

Certificar

Gold conciliado + medidas validadas.

04

Adoptar

Informes conectados al modelo oficial.

Los informes que aún calculen su propia venta se identifican como no oficiales hasta migrarlos. Toda diferencia con el transaccional debe quedar explicada y aceptada.

Estado en la entrega inicial · avances A01–A18 en el enlace superior

Plan y metas propuestos. La meta de ocho informes certificados no es un logro actual. Faltan responsables reales, presupuesto y validación del modelo destino.

Manuel Alberto Coy Benavides · Solución del proyecto Andina Market · Presentación asistida por Codex · Revisión 01 nov 2026

Revisar para corregir.
Explicar para que el equipo crezca.

QUÉ PIDE LA PRUEBA

Clasificar comentarios como bloqueantes, importantes o sugerencias, explicar por qué y proponer código corregido.

Comentario al autor del PR

La separación entre cálculo y envío permite ordenar la solución. Antes de integrar, necesitamos corregir seguridad y exactitud: ahora el total y el ticket no usan los mismos pedidos y los errores quedan ocultos. Te propongo los siguientes cambios con su motivo.

Prioridad	Hallazgo	Cambio y motivo
Bloqueante	Credenciales en código	Extraer secretos y usar mínimo privilegio; si la credencial del ejemplo fuera real, rotarla.
Bloqueante	SQL con f-string	Usar parámetros enlazados para separar datos e instrucciones.
Bloqueante	pedidos=[]	Eliminar estado compartido entre llamadas: contamina el denominador.
Bloqueante	Regla de venta inconsistente	Sumar solo entregados y usar el mismo conjunto para el ticket.
Bloqueante	Igualdad fecha/hora	Usar rango diario o la clave de fecha del agregado certificado.
Bloqueante	append y bucle de filas	Agregación SQL; evitar una API eliminada y copia reiterada de datos.
Bloqueante	except: pass	Registrar el error y terminar con fallo observable.

Prioridad	Hallazgo	Cambio y motivo
Bloqueante	División por cero	Sin pedidos, devolver ticket vacío.
Importante	Fecha y conexión	America/Bogota, día cerrado por defecto, conexión cerrada y sin ejecución al importar.
Importante	Fuente y pruebas	Consumir Gold, evitar SELECT * y probar día vacío, canales y llamadas repetidas.
Sugerencia	Mantenibilidad	Tipos, constantes, logging y envío de correo en un cambio separado.

▼ Ver versión corregida completa

```
"""Reporte diario de venta neta por canal (versión corregida del PR del analista junior).

Venta neta = valor_bruto - descuento de pedidos ENTREGADO, por fecha_pedido, en hora de Colombia.
Lee la tabla certificada gold.fact_venta_neta_diaria (la misma que alimenta Power BI), no la
tabla cruda dbo.pedidos: ahí los canales llegan con variantes ('app ', 'PAGINA WEB') y hay
pedidos duplicados, y el reporte daría una cifra distinta a la oficial.
"""
from __future__ import annotations

import logging
import os
import sys
from contextlib import closing
from dataclasses import dataclass
from datetime import date, datetime, timedelta
from zoneinfo import ZoneInfo

log = logging.getLogger(__name__)

CANALES = ("WEB", "APP", "TIENDA")
ZONA = ZoneInfo("America/Bogota")

# Todo el cálculo ocurre en el motor de base de datos: una consulta, parámetros enlazados
# (sin SQL armado con f-strings) y solo las columnas necesarias (sin SELECT *).
# Gold ya trae solo pedidos ENTREGADO, el canal normalizado y sin duplicados.
CONSULTA = """
SELECT c.canal,
       SUM(v.venta_neta)  AS total,
       SUM(v.num_pedidos) AS pedidos
FROM gold.fact_venta_neta_diaria v
```

```
    JOIN gold.dim_canal c ON c.canal_key = v.canal_key
    WHERE v.fecha_key = ?
    GROUP BY c.canal
    """


    @dataclass(frozen=True)
    class VentaCanal:
        canal: str
        total: int
        pedidos: int

        @property
        def ticket(self) -> int | None:
            """Ticket promedio; None (no 0) si no hubo pedidos, para no mostrar un dato falso."""
            return round(self.total / self.pedidos) if self.pedidos else None


    def ventas_por_canal(conn, dia: date) -> dict[str, VentaCanal]:
        """Venta neta y pedidos entregados de `dia`. Todos los canales aparecen, aunque no tengan ventas."""
        cur = conn.cursor()
        cur.execute(CONSULTA, (int(dia.strftime("%Y%m%d")),))  # fecha_key = yyyymmdd
        encontrados = {fila[0]: VentaCanal(fila[0], round(fila[1] or 0), int(fila[2])) for fila in cur.fetchall()}
        return {c: encontrados.get(c, VentaCanal(c, 0, 0)) for c in CANALES}


    def conectar():
        """La cadena de conexión viene del entorno (Key Vault / variable de entorno), nunca del código.
        Debe usar un usuario de solo lectura, no 'admin'."""
        import pyodbc
        return pyodbc.connect(os.environ["DWH_CONNECTION_STRING"], timeout=30)


    def dia_a_reportar(ahora: datetime | None = None) -> date:
        """Por defecto, el día anterior (cerrado) en hora de Colombia. Un informe de las 7:00 con el
        día en curso sale casi vacío. Supuesto a confirmar con quien pidió el reporte."""
        ahora = ahora or datetime.now(ZONA)
        return (ahora - timedelta(days=1)).date()


    def enviar_reporte(dia: date | None = None) -> None:
        dia = dia or dia_a_reportar()
        with closing(conectar()) as conn:
            resultado = ventas_por_canal(conn, dia)
        for canal, v in resultado.items():
            ticket = "sin pedidos" if v.ticket is None else f"{v.ticket:,}"
            log.info("Ventas %s %s: %s (%s pedidos) - ticket: %s", canal, dia, f"{v.total:,}", v.pedidos, ticket)
            # TODO: enviar el correo
        # No se captura nada aquí: si falla, la excepción sube, el proceso termina con error y la
        # herramienta que lo programa (Fabric, cron, Actions) alerta.
```

```
if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
    try:
        enviar_reporte()
    except Exception:
        log.exception("El reporte de ventas falló")
        sys.exit(1)
```

Abrir archivo fuente ↗

Pruebas del reporte corregido

Suma por canal, canal sin ventas, día sin pedidos, llamadas repetidas y fecha en hora de Colombia: cinco pruebas sobre una base SQLite en memoria.

Fuente: revisión detallada · pruebas.

Estado actual · A01–A18

Reporte corregido con pruebas y vista previa HTML local idempotente. Cargador SQL Server preparado; conexión ODBC, envío y destinatarios pendientes.

Primeras dos horas.
Contener, validar, comunicar.

> QUÉ PIDE LA PRUEBA
>
> El informe enviado a presidencia muestra 40 % menos venta que la real. Explicar la respuesta en máximo diez líneas.

01	7:30: aviso a gerencia y presidencia que la cifra está en revisión y fijo la próxima actualización a las 9:30.
02	Pauso envíos y refrescos; asigno un responsable y un canal único de comunicación.
03	Comparo fuente, Gold y reporte para ubicar dónde aparece la diferencia.
04	Reviso carga nocturna, frescura, volumen, cuarentena y conciliación; no asumo la causa del −40 %.
05	Contrasto despliegues recientes, cambios de filtros y definición del indicador.
06	Corrijo o revierto el cambio y reproceso; concilio antes de volver a publicar.
07	Envío la cifra validada a los mismos destinatarios, con explicación breve; si sigue abierta, informo estado y siguiente hito.
08	Incorporo una compuerta que bloquee publicación ante diferencias no explicadas o datos incompletos.
09	Añado una prueba de regresión, alerta temprana y verificación previa al envío ejecutivo.
10	En 48 horas documento causa y acciones con responsables y fechas, sin culpar a personas.

Estado en la entrega inicial · avances A01–A18 en el enlace superior

Respuesta propuesta al escenario del enunciado: diez puntos breves. No describe un incidente ocurrido ni acciones enviadas a terceros.

Manuel Alberto Coy Benavides · Solución del proyecto Andina Market · Presentación asistida por Codex · Revisión 01 nov 2026

Estándares claros.
Una conversación con hechos.

> QUÉ PIDE LA PRUEBA
>
> El desarrollador más antiguo rechaza estándares y sus cambios fallaron dos veces. Responder en máximo diez líneas.

01 Converso en privado para entender sus objeciones y reconocer su conocimiento del negocio.

02 Separo la discusión de estándares de los dos fallos de producción, que revisamos con hechos.

03 Identificamos juntos qué revisión o prueba habría prevenido cada fallo.

04 Le propongo liderar pruebas de negocio o revisar lógica crítica para aprovechar su experiencia.

05 Acordamos mínimos obligatorios: PR, pruebas y revisión antes de producción.

06 Distingo esos mínimos de las herramientas y prácticas que sí podemos ajustar.

07 Establezco 30 días de seguimiento con despliegues fallidos y reversiones como indicadores.

08 Facilito el cumplimiento con CI y protección de ramas; registro los acuerdos.

09 Si persiste el incumplimiento, escalo a mi responsable con evidencia e impacto.

10 Mantengo revisión obligatoria de sus cambios y el mismo estándar para todo el equipo.

Estado en la entrega inicial · avances A01–A18 en el enlace superior

Respuesta propuesta al escenario del enunciado: diez puntos breves. No describe un incidente ocurrido ni acciones enviadas a terceros.

Manuel Alberto Coy Benavides · Solución del proyecto Andina Market · Presentación asistida por Codex · Revisión 01 nov 2026

BONUS · CLASIFICACIÓN DE SOPORTE / Opcional · IA

Clasificar tickets.
Medir el resultado con honestidad.

QUÉ PIDE LA PRUEBA

Clasificar las descripciones por tipo de caso y explicar cómo medir la calidad.

01

2.125 tickets

Datos del caso

02

34 textos distintos

Una llamada por texto

03

17 tipos + otro

Salida JSON validada

04

Evaluación

Reglas vs dos LLM

EXACTITUD · 150 FRASES SINTÉTICAS



Resultados documentados para DeepSeek Flash y V4 Pro. Las reglas se ajustaron tras observar errores: el conjunto dejó de ser totalmente independiente.

LO QUE ESE 100 % NO PRUEBA

Desempeño en tickets nuevos reales

Las etiquetas fueron propuestas por IA y siguen pendientes de revisión humana. Los dos modelos empatan en un conjunto que puede ser demasiado fácil.

LECTURA CLAVE

Siguiente puerta: etiquetas humanas, muestra real reservada, F1 por clase, revisión de errores críticos y costo por ticket. El saldo medido de ~USD 0,03 cubre parte de las evaluaciones; no es un precio unitario.

Evidencia · Diseño y limitaciones de la evaluación · Resultados guardados

Medición	Aplicación
Exactitud y F1 macro / por tipo	Comparar globalmente y detectar clases críticas con mal desempeño.
Matriz de confusión	Distinguir errores entre clases similares y decisiones de alto impacto.
Muestra humana independiente	Separar desarrollo y evaluación; revisar etiquetas con dos anotadores.
Consistencia y respuestas inválidas	Repetir sin aprovechar caché y registrar fallos de formato.
Costo, privacidad e impacto	Medir por ticket; revisar texto libre y comprobar utilidad para enrutamiento.

Estado en la entrega inicial · avances A01–A18 en el enlace superior

Resultados de API conservados en resultados/ia. Las 34 etiquetas esperan revisión humana; las 150 frases nuevas son sintéticas. No se demuestra generalización a tickets reales nuevos.

Manuel Alberto Coy Benavides · Solución del proyecto Andina Market · Presentación asistida por Codex · Revisión 01 nov 2026